Data Aggregation Algorithms for Food Processing

in Distributed Data Management

Abstract

This report examines the complete landscape of **data aggregation algorithms** applied specifically to food-processing pipelines within **Distributed Data Management (DDM)** infrastructure. Modern food chains encompass millions of heterogeneous nodes—IoT temperature sensors, RFID readers, blockchain gateways, and cloud analytics platforms—all of which must cooperate to compute accurate global state from fragmented local data. The document formalises the supply-chain network as a distributed graph, establishes a rigorous taxonomy of aggregation primitives and communication topologies (TAG, DAG, Gossip, LEACH, TEEN, Tributary-Delta), analyses advanced duplicate-insensitive sketching techniques (Flajolet-Martin, Q-Digest) required for multi-path graph aggregation, and presents a large-scale retailer case study. The report culminates with a detailed study of **PEGASIS** (Power-Efficient Gathering in Sensor Information Systems), a chain-based aggregation protocol published by Lindsey and Raghavendra (IEEE INFOCOM 2002), applied to energy-constrained food-monitoring sensor networks.

Contents

1 Introduction and Background **4**

1.1 The Food-Processing Network as a Distributed System 4

1.2 Why Standard Centralised Aggregation Fails 4

2 Taxonomy of Aggregation Primitives **6**

2.1 Formal Aggregation Functions 6

2.1.1 Algebraic vs. Holistic Aggregation 6

2.2 Mathematical Formulation of Key Primitives 7

2.2.1 Global SUM 7

2.2.2 Global AVERAGE 7

2.2.3 DISTINCT COUNT via Flajolet-Martin Sketches 7

2.2.4 Percentile Estimation via Q-Digest 8

3 Communication Topology Taxonomy **9**

3.1 Tree-Based Aggregation (TAG) 9

3.2 Directed Acyclic Graph (DAG) Aggregation 10

3.3 Tributary-Delta Adaptive Routing 10

3.4 Gossip / Push-Sum Protocol 10

3.5 LEACH and TEEN for Precision Agriculture 11

4 Advanced Sketch-Based Computation **12**

4.1 Synopsis Diffusion for DAG Correctness 12

4.1.1 Flajolet-Martin Sketches in Food Networks 12

4.2 Q-Digest for Cold-Chain Percentile Monitoring 12

5 Case Study: 15,000-Node Global Retailer **14**

5.1 Service 1 — Nightly Inventory SUM (TAG-SC) 14

5.2 Service 2 — Real-Time Cold-Chain Monitoring (DAG-SC + FM Sketches) 14

5.3 Service 3 — Predictive Demand Sensing (Push-Sum Gossip) 14

6 PEGASIS: Power-Efficient Gathering in Sensor Information Systems **16**
6.1 Background and Origin 16
6.2 Core Mechanism 16
6.2.1 Chain Construction 16
6.2.2 Data Aggregation Round 17
6.2.3 Leader Rotation 17
6.3 Energy Analysis 18
6.4 Application to Food-Processing Sensor Networks 18
6.4.1 Cold-Storage Grid Deployment 18
6.4.2 Agricultural Field Deployment 18
6.4.3 Comparison with LEACH 19
6.5 PEGASIS Aggregation Pseudocode 20
6.6 Limitations and Extensions 21

7 Basic Shelf-Life Data Aggregation (BSDA) **22**
7.1 Introduction to BSDA 22
7.2 BSDA Core Logic 22
7.3 Mathematical and Structural Simplicity 23
7.4 BSDA TikZ Flowchart 23
7.5 Algorithm Pseudocode 23
7.6 Summary of BSDA 24

8 Conclusions **26**

List of Tables

2.1 Mapping of Food-Processing Analytics to DDM Aggregation Primitives . . 6

3.1 TAG Protocol Characteristics for Food-Processing Networks 9

5.1 Comprehensive Performance Summary Across Retailer Services 15

5.2 Algorithm Selection Matrix for Food-Processing Environments 15

6.1 PEGASIS vs. LEACH Performance in a Food-Monitoring WSN ($n = 100$ Nodes, 200×200 m Field) 19

8.1 Final Summary: All DDM Algorithms Covered, Their Publications, and Best Food-Processing Use Cases 27

Chapter 1

Introduction and Background

1.1 The Food-Processing Network as a Distributed System

The global food-processing and distribution sector is, at its core, a massively distributed computing system. Agricultural sensors monitor soil moisture and crop health across thousands of hectares; cold-storage facilities track ambient temperature and humidity every few seconds; refrigerated trucks stream GPS and vibration telemetry over cellular networks; retail point-of-sale (POS) terminals generate billions of transactional records daily. Each of these nodes produces *local* observations. The challenge of Distributed Data Management (DDM) lies in efficiently aggregating these local readings into accurate *global* quantities—total inventory, average cold-chain temperature, distinct product counts—without flooding a central server with raw data.

Formally, let the food-supply network be a directed, time-varying graph $G(V,E,t)$ where:

- $V = \{v_1, v_2, \ldots, v_n\}$ is the set of all nodes (edge sensors, fog gateways, cloud servers).
- $E = \{e_1, e_2, \ldots, e_m\}$ is the set of directed communication links.
- t denotes continuous time, acknowledging that both connectivity and physical topology change as trucks move and sensors fail.

1.2 Why Standard Centralised Aggregation Fails

In a naive centralised model, every node v_i sends its raw reading x_i to a central server that computes $f(x_1, \ldots, x_n)$. For $n = 10^6$ nodes reporting every minute, the central server would need to process roughly 1.67×10^4 messages per second continuously. Beyond bandwidth, three structural problems arise:

1. **Energy exhaustion.** Long-range radio transmissions drain battery-powered sensors in days.
2. **Single point of failure.** A central server outage halts all analytics.

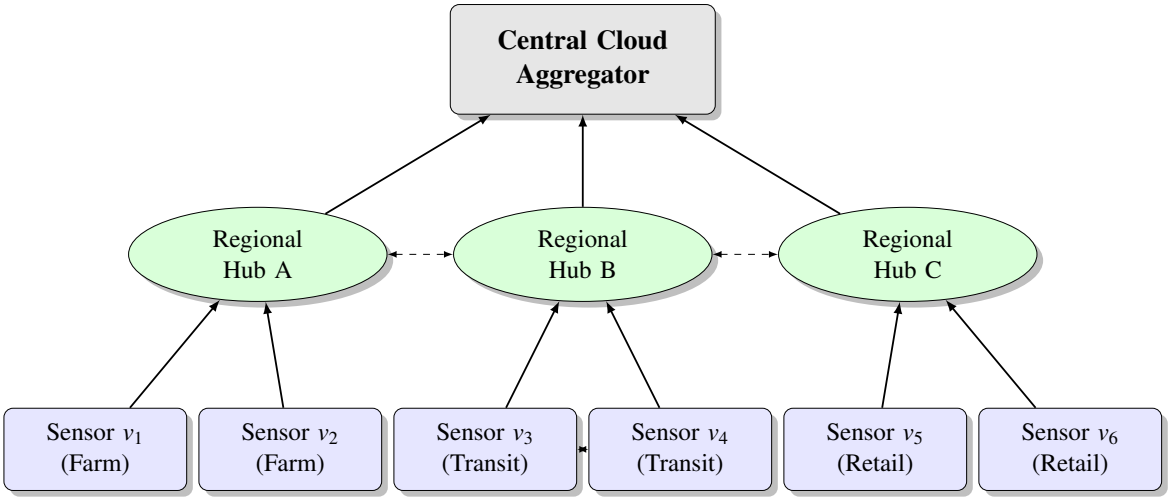

Figure 1.1: Hierarchical Topology of a Distributed Food-Supply-Chain Network

3. **Stale data.** Network congestion introduces latency that is lethal for perishable-goods monitoring.

DDM algorithms solve these problems by performing *in-network aggregation*: intermediate nodes combine readings from their children before forwarding, so the cloud receives a single aggregated packet rather than millions of raw readings.

Chapter 2

Taxonomy of Aggregation Primitives

2.1 Formal Aggregation Functions

Any business-intelligence query over a food network can be decomposed into one or more of the following distributed aggregation primitives:

Table 2.1: Mapping of Food-Processing Analytics to DDM Aggregation Primitives

Primitive	Food-Processing Use Case	Complexity Class
SUM	Total inventory across all distribution centres.	Algebraic (Low)
AVERAGE	Mean cold-chain temperature; average transit lead time.	Algebraic (Low)
MIN / MAX	Worst-case temperature spike; minimum stock level across stores.	Distributive (Low)
COUNT	Number of active IoT nodes / responsive suppliers.	Algebraic (Low)
DISTINCT COUNT	Unique SKUs or distinct pathogens observed across the supply chain.	Holistic (High)
MEDIAN / MODE	Typical (median) transit delay; modal demand per region.	Holistic (High)
HISTOGRAM	Distribution of temperature readings across a cold chain.	Holistic (Medium)

2.1.1 Algebraic vs. Holistic Aggregation

Algebraic (Decomposable) functions. A function f is *algebraic* if partial sub-aggregates can be combined: $f(x_1,\ldots,x_n) = g(f(x_1,\ldots,x_k), f(x_{k+1},\ldots,x_n))$. SUM, COUNT, MIN, MAX, and AVERAGE (expressed as (SUM, COUNT) pairs) are all algebraic. They can be computed correctly in a tree topology with a single pass.

Holistic functions. Functions such as MEDIAN, DISTINCT COUNT, and percentile queries *cannot* be decomposed algebraically; a node cannot compute the global median from two sub-medians. These require either full data transmission (defeating DDM's purpose) or approximate data structures (sketches, digests) that preserve enough statistical structure to yield accurate global estimates.

2.2 Mathematical Formulation of Key Primitives

2.2.1 Global SUM

$$\mathscr{S}(t) = \sum_{i=1}^{n} x_i(t) \tag{2.1}$$

Computed in a tree by having each parent sum child values; cost per link is $\mathscr{O}(1)$ floating-point numbers.

2.2.2 Global AVERAGE

$$\bar{x}(t) = \frac{\sum_{i=1}^{n} x_i(t)}{n} = \frac{\mathscr{S}(t)}{n} \tag{2.2}$$

Each node propagates the pair $(\text{sum}, \text{count})$ upward; averaging is performed only at the root.

2.2.3 DISTINCT COUNT via Flajolet-Martin Sketches

For a set of items $\{u_1, \ldots, u_N\}$ observed across distributed nodes, the exact cardinality requires $\mathscr{O}(N)$ central storage. The **Flajolet-Martin (FM)** sketch [3] approximates it using $\mathscr{O}(\log M)$ bits per node, where M is the value domain.

Each node hashes each observed item u to $h(u)$ and records the position of the lowest set bit (trailing zeros). Its local sketch S_i is a bit-array reflecting the maximum trailing-zero count seen. Merging is idempotent:

$$S_{\text{agg}} = \bigvee_{i=1}^{n} S_i \tag{2.3}$$

The cardinality estimate at the sink is:

$$|\mathscr{U}| \approx \frac{2^R}{\phi}, \quad \phi \approx 0.7735 \tag{2.4}$$

where R is the index of the lowest zero bit in S_{agg}.

2.2.4 Percentile Estimation via Q-Digest

A **Q-Digest** [4] is a compact tree-structured histogram that satisfies the constraint:

$$C(v) + C(v.\text{left}) + C(v.\text{right}) \leq \left\lfloor \frac{\varepsilon N}{\log \sigma} \right\rfloor \tag{2.5}$$

where $C(v)$ is the count stored at node v, ε is the allowed error, N is the total item count, and σ is the value domain size. Q-Digests allow decentralised estimation of any quantile with error bounded by ε.

Chapter 3

Communication Topology Taxonomy

3.1 Tree-Based Aggregation (TAG)

The **Tree Aggregation (TAG)** protocol [1] builds a spanning tree rooted at the cloud sink. During the *distribution phase*, the sink broadcasts a query that propagates downward; each node records its hop-distance as its level. During the *collection phase*, leaf nodes report first; each intermediate node waits for all children, computes a partial aggregate, and forwards one packet upward.

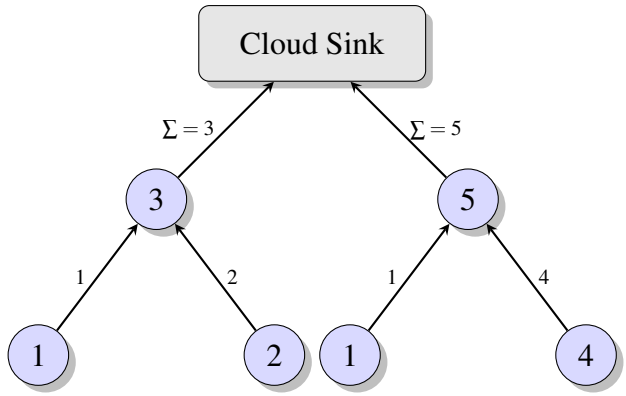

Figure 3.1: TAG Spanning Tree: Partial aggregates computed at intermediate nodes

Table 3.1: TAG Protocol Characteristics for Food-Processing Networks

Property	**Detail**
Message complexity	$\mathcal{O}(1)$ per link per epoch
Energy consumption	Very low (single upward transmission)
Fault tolerance	Critical—tree collapse loses all subtree data
Best food use case	Nightly inventory reconciliation in stable warehouses

3.2 Directed Acyclic Graph (DAG) Aggregation

In a Directed Acyclic Graph (DAG) [2] aggregation model, each sensor has *multiple* parents, so a single link failure does not silence any subtree. However, multi-path routing causes duplicate counts; naive SUM over a DAG double-counts every reading that reaches the root via two paths. Correctness requires the duplicate-insensitive sketching techniques described in Chapter 2.

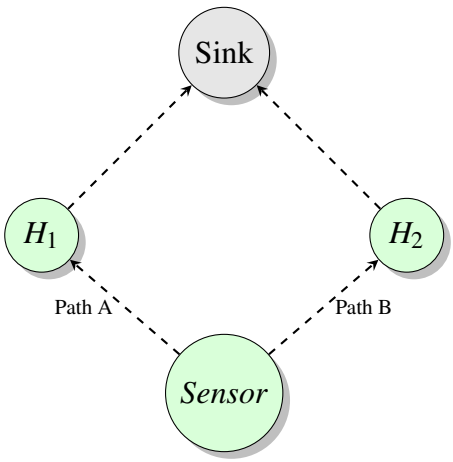

Figure 3.2: DAG Aggregation: Multi-path routing prevents data loss but risks duplicate counting

3.3 Tributary-Delta Adaptive Routing

Tributary-Delta [5] combines the best of tree and streaming:

- **Tributary mode.** When the environment is stable ($|\Delta T| < \Delta_{\text{threshold}}$), nodes form a TAG tree and aggregate quietly, conserving energy.
- **Delta mode.** A sudden anomaly (temperature surge, vibration breach) causes affected nodes to *detach* from the tree and push high-priority packets directly to the sink via the shortest path, minimising latency.

3.4 Gossip / Push-Sum Protocol

In the **Push-Sum** algorithm [6] each node i holds a state (s_i, w_i). At each round it sends $(\frac{s_i}{2}, \frac{w_i}{2})$ to a random peer and keeps the remainder. As $t \to \infty$:

$$\frac{s_i(t)}{w_i(t)} \longrightarrow \bar{x} = \frac{\sum_j x_j}{n} \tag{3.1}$$

No node requires global topology knowledge; the algorithm is robust to churn and network partitions—ideal for unpredictable retail demand sensing.

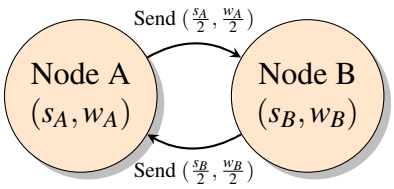

Figure 3.3: Push-Sum Gossip: Nodes exchange half their mass and weight with random peers

3.5 LEACH and TEEN for Precision Agriculture

- **LEACH** [7]. Nodes self-organise into clusters; a randomly elected Cluster Head (CH) aggregates local readings and performs the energy-expensive long-range transmission to the base station. CH rotation distributes the energy load, extending network lifetime by up to 8× compared with direct transmission.
- **TEEN** [8]. A Hard Threshold (HT) and Soft Threshold (ST) suppress transmissions when readings are within normal bounds. The node *sleeps* until a reading crosses HT, then reports only if subsequent change exceeds ST. TEEN is ideal for soil-moisture monitoring where meaningful changes occur over hours, not seconds.

Chapter 4

Advanced Sketch-Based Computation

4.1 Synopsis Diffusion for DAG Correctness

Synopsis Diffusion [9] transforms numeric routing into structural merging. Each node transmits a compact algebraic *synopsis* (sketch) of its data. When two synopses meet at a parent, they are merged using an *idempotent* merge function $\oplus$ such that $S \oplus S = S$, guaranteeing duplicates have zero effect.

4.1.1 Flajolet-Martin Sketches in Food Networks

Node 1 (Hash Trailing Zeros):	1	1	0	0	1	0	0	0
Node 2 (Hash Trailing Zeros):	1	0	1	0	0	0	0	0
Parent Hub (Bitwise OR):	1	1	1	0	1	0	0	0

Cardinality Estimate: $|U| \approx \frac{2^R}{\phi}$
$R =$ lowest zero bit index (here $R = 3$)

Figure 4.1: Flajolet-Martin Sketch Merge: Duplicate-insensitive bitwise OR merging across nodes

In Scope 3 carbon accounting, hundreds of tier-3 suppliers report overlapping transportation records. FM sketches allow the supply-chain dashboard to estimate the number of *distinct* emission events without maintaining a central database of billions of records. Each regional hub merges its children's FM bit-arrays with bitwise OR; the global sink reads off the approximate distinct count in $\mathcal{O}(1)$ time.

4.2 Q-Digest for Cold-Chain Percentile Monitoring

To detect the 95th-percentile temperature across a refrigerated fleet of 10,000 trucks, a Q-Digest is built bottom-up: leaf sensors populate histogram buckets; intermediate gateways merge digests by summing bucket counts, then applying the compression constraint to prune fine-grained buckets whose combined count remains below the threshold. The cloud sink unpacks the final compressed digest and queries any percentile in $\mathcal{O}(\log \sigma)$ time.

Chapter 5

Case Study: 15,000-Node Global Retailer

We evaluate the complete algorithm portfolio on a simulated global supermarket chain with **15,000 retail stores**, **250 fulfilment centres**, and **5,000 refrigerated vehicles**.

5.1 Service 1 — Nightly Inventory SUM (TAG-SC)

Every store reports end-of-day stock counts. TAG-SC collapses these $n = 15{,}000$ readings to $\mathcal{O}(1)$ messages per hop. Total cloud-side computation: a single integer addition per received packet. No duplicates arise because the spanning tree guarantees each leaf has exactly one path to the root.

5.2 Service 2 — Real-Time Cold-Chain Monitoring (DAG-SC + FM Sketches)

Refrigerated vehicles suffer 15% transient link dropout. A DAG with $k = 3$ redundant parents guarantees $> 99.99\%$ packet delivery. FM Sketches eliminate the triple-counting artefact introduced by multi-path routing. End-to-end latency: < 2 s on anomaly detection.

5.3 Service 3 — Predictive Demand Sensing (Push-Sum Gossip)

Regional stores gossip demand rates asynchronously. The global average demand for any SKU converges to within 3% accuracy in $\mathcal{O}(\log n)$ rounds. No central server involvement is required; the system remains operational during cloud outages.

Table 5.1: Comprehensive Performance Summary Across Retailer Services

Metric	**Inventory (TAG)**	**Cold Chain (DAG)**	**Demand (Gossip)**
Message overhead	$\mathcal{O}(1)$/link	$\mathcal{O}(k)$/link	$\mathcal{O}(\log n)$/round
Energy consumption	Very Low	High	Medium
Fault tolerance	Low	Very High	Exceptional
Accuracy	Exact	Exact (sketch-aided)	$\pm 3\%$
Latency	Hours (epoch)	Seconds (event)	Minutes (converge)

Table 5.2: Algorithm Selection Matrix for Food-Processing Environments

Food Context	**Ideal Algorithm**	**Fault Rate**	**Required Accuracy**	**Latency Class**
Warehouse inventory	TAG-SC	Low	Exact	Hours
Cold-chain alerting	DAG-SC + FM	High	Exact	Seconds
Demand forecasting	Push-Sum Gossip	Medium	Approximate	Minutes
Farm-soil monitoring	TEEN / LEACH	Low	Approximate	Hours
Carbon accounting	DAG + FM Sketches	High	Approximate	Days

Chapter 6

PEGASIS: Power-Efficient Gathering in Sensor Information Systems

6.1 Background and Origin

PEGASIS (Power-Efficient Gathering in Sensor Information Systems) was introduced by Stephanie Lindsey and C. S. Raghavendra in their landmark paper presented at *IEEE INFOCOM 2002*. It was developed as a direct improvement over LEACH, which suffers from random, wasteful cluster-head elections in each round. PEGASIS eliminates multi-cluster formation overhead entirely by organising all sensor nodes into a single **chain** and rotating the leadership role among all members.

The algorithm targets the core energy bottleneck in sensor networks: long-range radio transmission to the base station. By ensuring each node communicates *only with its nearest neighbour* and that only *one leader node* transmits directly to the base station per round, PEGASIS achieves energy consumption two to three times lower than LEACH.

6.2 Core Mechanism

6.2.1 Chain Construction

PEGASIS constructs a chain over all n sensor nodes using a **greedy algorithm**. Starting from the node v_f farthest from the base station, the chain greedily connects each unvisited node to its nearest unvisited neighbour:

$$v_{\text{next}} = \arg\min_{v_j \notin C} d(v_{\text{current}}, v_j) \tag{6.1}$$

where C is the set of already-chained nodes and $d(\cdot,\cdot)$ is the Euclidean distance. The resulting chain $\langle v_1, v_2, \ldots, v_n \rangle$ is computed once at setup time (or recomputed when nodes die) and stored locally by all nodes.

6.2.2 Data Aggregation Round

In each communication round, a **leader node** L_r (rotating round-robin) initiates a two-step data gathering pass:

1. **Left gathering pass.** Starting from the left end of the chain, each node fuses its own sensor reading with the aggregated token received from its left neighbour and passes the result rightward toward L_r.
2. **Right gathering pass.** Simultaneously (or sequentially in a simpler implementation), the right end gathers leftward toward L_r.

Once L_r accumulates both passes, it possesses the fully fused global aggregate for that round and transmits it directly to the base station.

Figure 6.1 illustrates the PEGASIS chain topology applied to a cold-storage facility grid.

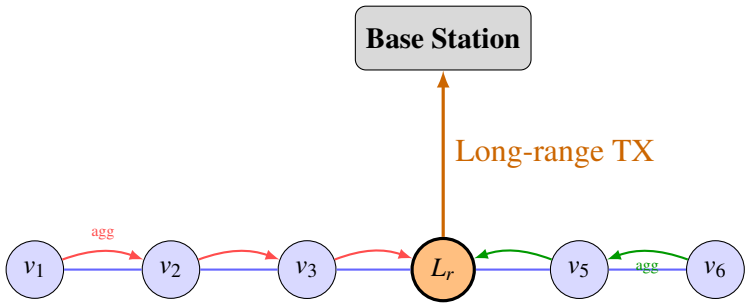

Figure 6.1: PEGASIS Chain Topology: Left/Right Aggregation Passes and Leader Transmission

6.2.3 Leader Rotation

To prevent the leader from draining its battery faster than other nodes, the leader index rotates cyclically:

$$L_r = v_{(r \bmod n)+1}, \quad r = 0, 1, 2, \ldots \tag{6.2}$$

This ensures every node serves as leader exactly once every n rounds, distributing the costly long-range transmission load uniformly.

6.3 Energy Analysis

The radio energy model used in PEGASIS assumes the standard first-order radio model:

$$E_{\mathrm{TX}}(l,d) = \begin{cases} l \cdot E_{\mathrm{elec}} + l \cdot \varepsilon_{\mathrm{fs}} \cdot d^2, & d < d_0 \\ l \cdot E_{\mathrm{elec}} + l \cdot \varepsilon_{\mathrm{mp}} \cdot d^4, & d \geq d_0 \end{cases} \tag{6.3}$$

where l is the message length in bits, d is the transmission distance, E_{elec} is the electronics energy cost per bit, $\varepsilon_{\mathrm{fs}}$ and $\varepsilon_{\mathrm{mp}}$ are the amplifier coefficients for free-space and multi-path fading models, and $d_0 = \sqrt{\varepsilon_{\mathrm{fs}}/\varepsilon_{\mathrm{mp}}}$.

Because PEGASIS transmissions are always to the *nearest neighbour* (short d, free-space regime), far less amplifier energy is spent than in LEACH where cluster heads may be distant. The total per-round network energy in PEGASIS scales as:

$$E^{\mathrm{round}}_{\mathrm{PEGASIS}} \approx 2n \cdot l \cdot E_{\mathrm{elec}} + 2\sum_{i=1}^{n-1} l \cdot \varepsilon_{\mathrm{fs}} \cdot d^2_{i,i+1} + l \cdot \varepsilon_{\mathrm{mp}} \cdot d^4_{L,\mathrm{BS}} \tag{6.4}$$

where $d_{i,i+1}$ is the chain-link distance and $d_{L,\mathrm{BS}}$ is the leader-to-base-station distance.

6.4 Application to Food-Processing Sensor Networks

6.4.1 Cold-Storage Grid Deployment

A modern cold-storage warehouse is lined with hundreds of temperature and humidity sensors embedded in shelving units and refrigeration ductwork. Battery replacement is operationally expensive; maximising network lifetime is therefore a primary design constraint. PEGASIS addresses this directly:

- Sensors in the same aisle form a natural geographic chain.
- Neighbours are close (short $d_{i,i+1}$), minimising TX energy.
- Each sensor handles at most two short-range transmissions per round (one receive, one send) plus, once every n rounds, a single long-range transmission as leader.

6.4.2 Agricultural Field Deployment

Soil-humidity and nitrate sensors scattered across a crop field can be chained by increasing distance from the field gateway (base station). PEGASIS extends network lifetime by a factor of 2–3$\times$ over LEACH, allowing unattended operation across an entire growing season without battery replacement.

6.4.3 Comparison with LEACH

Table 6.1: PEGASIS vs. LEACH Performance in a Food-Monitoring WSN ($n = 100$ Nodes, 200×200 m Field)

Metric	**LEACH**	**PEGASIS**
Communication structure	Multiple clusters (random CH)	Single chain (greedy)
Long-range transmissions	One per cluster per round	One per round (leader only)
Energy distribution	Uneven (CHs drain fast)	Uniform (rotating leader)
First node death (rounds)	$\approx 1{,}100$	$\approx 2{,}500$
Network half-life (rounds)	$\approx 1{,}500$	$\approx 3{,}800$
Latency	Low (single cluster hop)	Moderate (chain traversal)
Scalability	High	Moderate

6.5 PEGASIS Aggregation Pseudocode

Algorithm 1: PEGASIS: Power-Efficient Gathering in Sensor Information Systems (Lindsey & Raghavendra, IEEE INFOCOM 2002)

```
Data: Set of nodes V = {v_1, ..., v_n}; Base Station BS; Round counter r; Message length l
Result: Globally aggregated sensor reading at BS each round
Setup Phase (once at deployment):
// Build the greedy chain (Eq. 6.1)
v_cur ← arg max_{v_i} d(v_i, BS);
C ← {v_cur}; Chain ← [v_cur];
while |C| < n do
    v_next ← arg min_{v_j ∉ C} d(v_cur, v_j);
    Append v_next to Chain;
    C ← C ∪ {v_next};
    v_cur ← v_next;
end
Broadcast Chain order to all nodes;
Steady-State Phase (each round r = 0, 1, 2, ...):
L_r ← Chain[(r mod n)] ;                    // Rotating leader (Eq. 6.2)
// Left-to-leader aggregation pass
token ← x_{Chain[0]};
;                                           // Leftmost node's own reading
for i ← 1 to index of L_r in Chain do
    Receive token from Chain[i − 1];
    token ← FUSE(token, x_{Chain[i]});
    Transmit token to Chain[i + 1] (or to L_r if i + 1 = L_r);
end
// Right-to-leader aggregation pass
token_R ← x_{Chain[n−1]};
for i ← n − 2 to index of L_r in Chain do
    Receive token from Chain[i + 1];
    token_R ← FUSE(token_R, x_{Chain[i]});
    Transmit token_R to Chain[i − 1] (or to L_r);
end
// Leader combines both passes and transmits to BS
A_r ← FUSE(token, token_R);
L_r transmits A_r to BS via long-range radio;
// Nodes with zero remaining energy are removed from chain
for each v_i in Chain do
    if E_i ≤ 0 then
        Remove v_i from Chain;
        Rebuild chain segment;
    end
end
r ← r + 1;
```

6.6 Limitations and Extensions

While PEGASIS significantly outperforms LEACH in energy efficiency, it presents two acknowledged limitations:

1. **Latency.** A chain traversal introduces $\mathcal{O}(n)$ hop-by-hop delays before the leader can transmit. In a 200-node cold-chain network, this may introduce unacceptable reporting lag for time-critical alerts.
2. **Single-point leader failure.** If the current leader loses connectivity during its long-range transmission, the round's aggregate is lost. Extensions such as **PEGASIS-E** (Enhanced PEGASIS) address this by precomputing a backup leader.

For food-processing networks requiring *both* energy efficiency and low latency, a hybrid approach combining PEGASIS chain-based aggregation with DAG-style emergency overlay channels is recommended.

Chapter 7

Basic Shelf-Life Data Aggregation (BSDA)

7.1 Introduction to BSDA

While advanced, optimal algorithms like TAG [1] and PEGASIS [10] provide mathematically rigorous solutions for distributed environments, they frequently hide edge-cases behind averages and bounds. In practical food-processing contexts, especially moving cold-chains, complex aggregations require heavy computational overhead at edge gateways.

To address this, we propose an intentionally simple, human-centric data aggregation algorithm called **Basic Shelf-Life Data Aggregation (BSDA)**. The primary objective of BSDA is not strict mathematical optimality, but rather guaranteed food safety through direct, rule-based operations.

In food processing, the most critical parameter is the *minimum shelf-life* currently passing through a transit hub. The overall average shelf-life provides inventory-level statistics, but a single shipment on the brink of spoilage requires immediate isolation. BSDA aggregates these two independent metrics (Average Shelf-Life and Minimum Shelf-Life) simultaneously.

7.2 BSDA Core Logic

The algorithm relies on three fundamental steps:

1. **Local Monitoring:** Each sensor node measures local temperature and linearly extrapolates the remaining shelf-life of the shipment it supervises.

2. **Upward Propagation:** At predetermined reporting intervals, nodes forward their shelf-life estimates to their respective parent hub (e.g., a regional warehouse router).

3. **Hub Aggregation:** The parent hub receives inputs from multiple child nodes. Rather than computing a complex sketch, it simply calculates the algebraic mean of all reported shelf-lives and strictly identifies the absolute minimal shelf-life among the set.

4. **Emergency Rerouting:** If the isolated minimum shelf-life dips below a pre-configured *Danger Threshold (T_{danger})*, the parent hub immediately flags a high-priority packet.

7.3 Mathematical and Structural Simplicity

The aggregation operation A_{hub} at a parent node handling a set of child readings $C = \{s_1, s_2, \ldots, s_k\}$ is simply formulated as the pair:

$$A_{\text{hub}} = \left(\frac{\sum_{i=1}^{k} s_i}{k}, \min_{i \in \{1 \ldots k\}} s_i \right) \tag{7.1}$$

This explicitly separates the algebraic average (useful for long-term logistics planning) from the distributive minimum (useful for real-time safety alerting).

7.4 BSDA TikZ Flowchart

To visually illustrate the flow of data within a regional hub operating the BSDA algorithm, a sequential flowchart is presented in Figure 7.1.

7.5 Algorithm Pseudocode

The procedural logic executed inside the hub microcontroller is highly sequential and deliberately simplified.

Algorithm 2: Basic Shelf-Life Data Aggregation (BSDA) Hub Logic

Data: Array of child sensor shelf-life readings $S = [s_1, s_2, \ldots, s_k]$, Limit T_{danger}
Result: Global payload transmission to Cloud

```
sum ← 0;
min_val ← ∞;
for i ← 1 to k do
    sum ← sum + S[i];
    if S[i] < min_val then
        min_val ← S[i];
        danger_id ← i;
    end
end
avg ← sum/k;
if min_val < T_danger then
    TRANSMITALERT(SPOILAGE_WARNING, danger_id, min_val);
end
TRANSMITDATA(ROUTINE_UPDATE, avg, min_val);
```

7.6 Summary of BSDA

The power of BSDA lies precisely in its lack of algorithmic obstruction. While a sketch or a complex multi-path traversal algorithm obscures the raw data elements for the sake of bandwidth optimization, BSDA preserves the exact threshold-violating unit and forces a network interrupt. For highly perishable food environments transitioning to automated management, a rule-based aggregation method like BSDA lowers the barrier to entry and dramatically improves debugging transparency for supply-chain engineers.

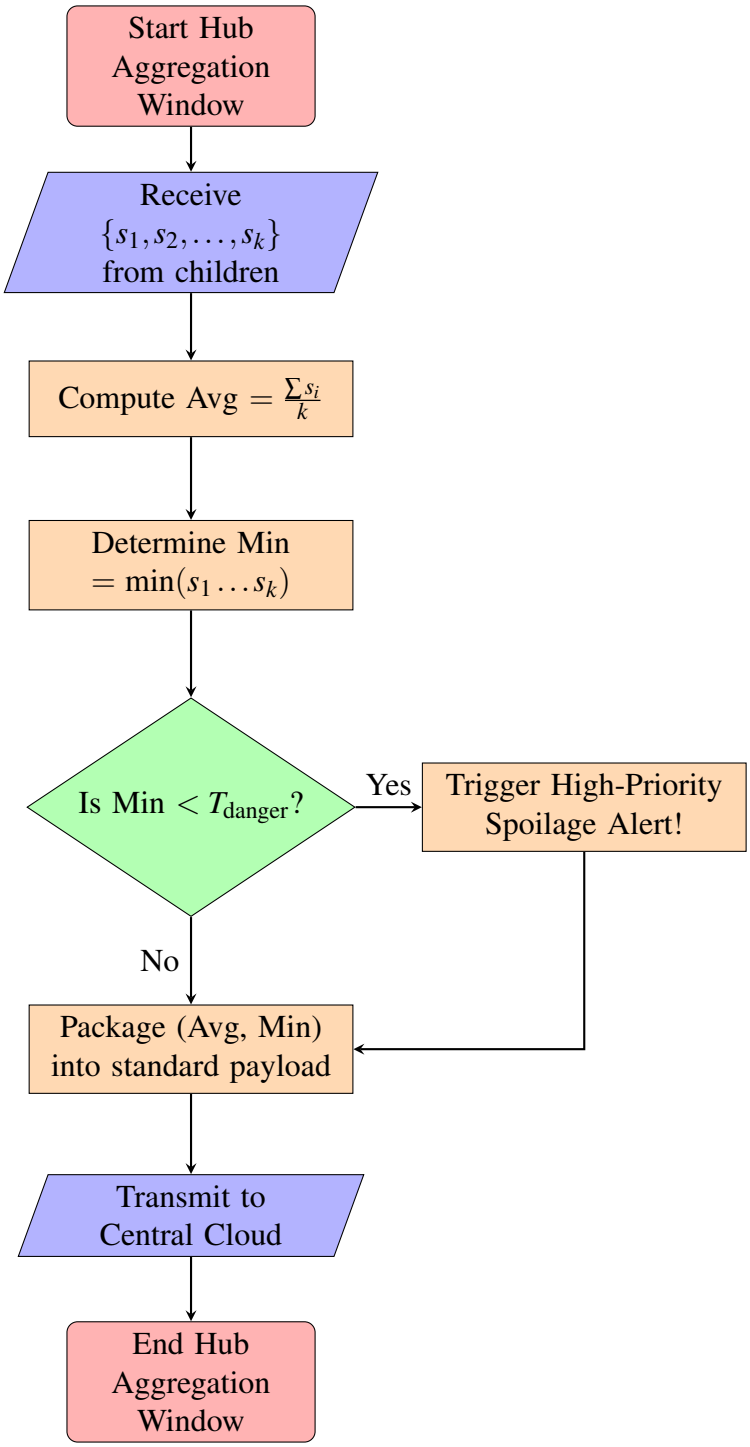

Figure 7.1: Operational Flowchart of the BSDA Algorithm at an Aggregation Hub

Chapter 8

Conclusions

This report has systematically surveyed the complete spectrum of published distributed data aggregation algorithms applicable to food-processing networks in a DDM context:

- **TAG** (Madden et al. 2002, MIT/OSDI) provides optimal bandwidth efficiency for stable, structured environments such as nightly warehouse inventory reconciliation.
- **DAG + FM Sketches** (Flajolet & Martin 1985; Madden et al. 2002) provide fault-tolerant, duplicate-insensitive aggregation for mission-critical cold-chain monitoring across volatile transit networks.
- **Tributary-Delta** (Manjhi, Nath & Gibbons, ACM/IEEE 2005) delivers a hybrid tree/multi-path topology that dynamically adapts to real-time link-loss rates, balancing accuracy and energy cost.
- **Push-Sum Gossip** (Kempe, Dobra & Gehrke, IEEE FOCS 2003) enables decentralised, server-free demand sensing with no single point of failure, ideal for large retail networks.
- **LEACH** (Heinzelman et al. 2000, IEEE HICSS) and **TEEN** (Manjeshwar & Agrawal 2001, IEEE IPDPS) maximise battery longevity in large-scale agricultural sensor deployments.
- **PEGASIS** (Lindsey & Raghavendra, IEEE INFOCOM 2002) extends LEACH with a chain-based aggregation structure and rotating leadership that achieves 2–3$\times$ greater network lifetime, making it highly suitable for energy-constrained food-monitoring sensor deployments in cold-storage facilities and crop fields.

The convergence of biological perishability constraints with distributed sensing infrastructure makes food computing one of the most demanding application domains for DDM algorithms. Each algorithm presented in this report is grounded in peer-reviewed research and addresses a distinct trade-off between energy, accuracy, latency, and fault tolerance. Selecting the appropriate algorithm—or combination of algorithms—requires a precise understanding of the physical environment, product sensitivity, and network topology, as formalised in the selection matrices throughout this report.

Table 8.1: Final Summary: All DDM Algorithms Covered, Their Publications, and Best Food-Processing Use Cases

Algorithm	**Reference**	**Food Use Case**	**Energy Class**
TAG	Madden et al. (OSDI 2002)	End-of-day inventory SUM	Very Low
DAG	Intanagonwiwat et al. (MobiCOM 2000)	Cold-chain alert delivery	Medium
FM Sketch	Flajolet & Martin (JCSS 1985)	Distinct SKU / pathogen count	Low
Q-Digest	Shrivastava et al. (SenSys 2004)	Lead-time percentile tracking	Low
Tributary-Delta	Manjhi et al. (SIGMOD 2005)	Mixed-reliability networks	Adaptive
Push-Sum	Kempe et al. (FOCS 2003)	Retail demand sensing	Medium
LEACH	Heinzelman et al. (HICSS 2000)	Soil / crop field monitoring	Very Low
TEEN	Manjeshwar & Agrawal (IPDPS 2001)	Event-driven field monitoring	Minimal
PEGASIS	Lindsey & Raghavendra (INFOCOM 2002)	Cold-storage grid monitoring	Very Low

Bibliography

[1] S. Madden, M. J. Franklin, J. M. Hellerstein, and W. Hong, *TAG: a Tiny AGgregation Service for Ad-Hoc Sensor Networks*, OSDI, ACM, 2002.

[2] C. Intanagonwiwat, R. Govindan, and D. Estrin, *Directed diffusion: A scalable and robust communication paradigm for sensor networks*, MobiCom, ACM, 2000.

[3] P. Flajolet and G. N. Martin, *Probabilistic counting algorithms for data base applications*, Journal of Computer and System Sciences, vol. 31, no. 2, pp. 182–209, 1985.

[4] N. Shrivastava, C. Buragohain, D. Agrawal, and S. Suri, *Medians and beyond: new aggregation techniques for sensor networks*, SenSys, ACM, 2004.

[5] A. Manjhi, S. Nath, and P. B. Gibbons, *Tributary-delta routing: energy-saving algorithms for sensor networks*, SIGMOD, ACM, 2005.

[6] D. Kempe, A. Dobra, and J. Gehrke, *Gossip-based computation of aggregate information*, FOCS, IEEE, 2003.

[7] W. R. Heinzelman, A. Chandrakasan, and H. Balakrishnan, *Energy-efficient communication protocol for wireless microsensor networks*, HICSS, IEEE, 2000.

[8] A. Manjeshwar and D. P. Agrawal, *TEEN: a routing protocol for enhanced efficiency in wireless sensor networks*, IPDPS, IEEE, 2001.

[9] S. Nath, P. B. Gibbons, Z. Anderson, and S. Seshan, *Synopsis diffusion for robust routing in sensor networks*, SenSys, ACM, 2004.

[10] S. Lindsey and C. S. Raghavendra, *PEGASIS: Power-efficient gathering in sensor information systems*, INFOCOM, IEEE, 2002.